

Uygulanan İyileştirmeler Raporu

Olusturulma: 14 Nisan 2026 19:49 Kaynak: docs/audit Format: Tekil rapor

Uygulanan İyileştirmeler Raporu

Tarih: 2026-04-14 Kapsam: Audit raporlarında belirlenen ürün, UX/UI ve frontend mimarisi eksiklerine göre uygulanan kalıcı iyileştirmeler. Hedef okuyucu: Ürün sahibi, işletmeci, operasyon yöneticisi ve teknik ekip.

1. Yönetici Özeti

Bu çalışmada restoran POS uygulamasında daha önce raporlanan sorunlar içinden kullanıcıya doğrudan yansıyan ve ürün kalitesini düşüren alanlar ele alındı. Amaç hızlı bir geçici düzeltme yapmak değil; ödeme, sipariş, masa ve ayarlar ekranlarında daha tutarlı, güvenilir ve sürdürülebilir bir temel oluşturmaktır.

Alan	Önceki Durum	Yeni Durum	Kullanıcıya Etkisi
Ödeme kuralları	Farklı ekranlarda ayrı ayrı hesaplanıyordu	Tek merkezden yönetiliyor	Ödeme, hızlı ödeme ve masa durumları daha tutarlı
Sipariş aksiyonları	Kaydetme, ödeme açma ve satır düzenleme koşulları ekran içine dağılmıştı	Ortak karar fonksiyonlarına taşındı	Hatalı veya zamansız aksiyon riski azaldı
Onay pencereleri	Bazı yerlerde tarayıcıya ait kaba onay kutuları kullanılıyordu	Uygulama içi ortak onay penceresi kullanılıyor	Daha profesyonel ve daha anlaşılır deneyim
Ayarlar ekranları	Çıkış, silme, pasifleştirme gibi kritik işlemler tutarsızdı	Aynı onay standardına bağlandı	Yönetim tarafında güven ve tutarlılık arttı
API altyapısı	HTTP iletişimi ve domain endpointleri aynı dosyada büyüyordu	HTTP çekirdeği ayrıldı	Sonraki geliştirmeler için daha sağlam temel oluştu

2. Neler İyileştirildi?

2.1 Ödeme ve Masa Durumu Daha Güvenilir Hale Getirildi

Ödeme ekranı, hızlı ödeme ekranı ve masa ekranı daha önce "bu sipariş ödendi mi?", "ne kadar kaldı?", "masa kapatılabilir mi?" gibi soruları kendi içinde ayrı ayrı cevaplıyordu. Bu durum ileride bir ekranın doğru, diğerinin yanlış davranmasına yol açabilecek bir riskti.

Yeni yapıda ödeme durumu tek merkezden hesaplanıyor.

İyileştirme	Açıklama	Kazanç
Toplam ödeme hesabı ortaklaştırıldı	Sipariş toplamı, ödenen tutar ve kalan tutar aynı yardımcı yapıdan geliyor	Ekranlar arası tutarsızlık riski azaldı
Tam ödeme kontrolü ortaklaştırıldı	"Tam ödendi" kararı tek yerden veriliyor	Masa kapatma ve hızlı ödeme davranışı daha güvenli
Paket sipariş ödeme etiketi ortaklaştırıldı	Ödendi, kısmi ödendi, ödenmedi gibi durumlar aynı mantıkla gösteriliyor	Paket ve masa siparişi dili daha tutarlı

Etkilenen kullanıcı deneyimi:

- Kasiyer hızlı ödeme yaptıktan sonra masa durumunun yanlış görünme riski azaldı.
- Paket siparişlerde ödeme etiketi daha tutarlı hale geldi.
- Masa kapatma gibi kritik işlemler ödeme durumuyla daha net bağlandı.

3. Sipariş Ekranında Karar Mantığı Toparlandı

Sipariş ekranı POS uygulamasının en yoğun kullanılan alanı. Bu ekranda "kaydet", "ödeme al", "ürün düzenle", "ürün iptal et" gibi kararlar daha önce ekran kodunun içine dağılmış durumdaydı.

Bu turda bu kararların bir kısmı merkezi ve okunabilir kurallara taşındı.

Kural	Önceki Risk	Yeni Yaklaşım
Sipariş kaydedilebilir mi?	Sepet boşken veya paket siparişte müşteri yokken dağınık kontroller vardı	Tek karar fonksiyonuyla kontrol ediliyor
Ödeme ekranı açılabilir mi?	Kaydedilmemiş ürün varken ödeme açma riski ekran içine gömülüydü	Merkezi aksiyon kuralına taşındı
Ürün satırı düzenlenebilir mi?	Satır durumu ve sipariş durumu kontrolleri dağınıktı	Ortak policy fonksiyonuyla yönetiliyor
Ürün iptal edilebilir mi?	Yetki ve satır durumu kontrolleri ekranda karışıyordu	Daha net bir karar katmanına alındı

Bu iyileştirme kullanıcıya nasıl yansır?

- Kaydedilmemiş ürün varken yanlışlıkla ödeme akışına girme riski azalır.
- Kapalı siparişlerde yanlış düzenleme aksiyonları daha güvenli engellenir.
- Paket siparişlerde müşteri zorunluluğu daha anlaşılır şekilde korunur.

4. Uygulama İçi Onay Deneyimi Standartlaştırıldı

Audit raporlarında "amatör görünen alanlar" içinde en net sorunlardan biri native tarayıcı onay kutularıydı. Bunlar görsel olarak uygulamanın geri kalanından kopuk, açıklama açısından zayıf ve yoğun POS kullanımında güven vermeyen bir deneyim oluşturuyordu.

Bu turda ortak bir onay penceresi ve onu yöneten ortak kullanım yapısı eklendi.

Nerede Kullanılıyor?	Örnek Aksiyon	Yeni Davranış
İşletme ayarları	Kaydetmeden çıkış, değişiklikleri atma	Açıklamalı uygulama içi onay
Ekran ayarları	Kaydetmeden çıkış	Açıklamalı uygulama içi onay
Menü yönetimi	Kategori silme, ürün kaldırma, toplu pasifleştirme	Standart tehlikeli işlem onayı
Yazıcı ayarları	Kaydetmeden çıkış, varsayılanlara dönme, pasifleştirme	Daha kontrollü onay akışı
Kullanıcı yönetimi	Kullanıcı pasifleştirme	Net etki açıklamasıyla onay
Rezervasyonlar	Rezervasyon silme	Standart silme onayı
Stok	Stok kalemi silme	Standart silme onayı
Salon bölgeleri	Bölge silme	Risk açıklamalı onay

Kullanıcı açısından fark:

- Kritik işlemlerde "ne olacak?" sorusu daha net cevaplanıyor.
- Uygulama artık tarayıcı penceresi gibi değil, kendi içinde tutarlı bir ürün gibi davranıyor.
- Silme, pasifleştirme ve çıkış işlemleri daha güvenli hissediliyor.

5. Ayarlar ve Yönetim Ekranlarında Profesyonellik Artırıldı

Ayarlar ekranları işletme sahibi veya yönetici tarafından kullanılıyor. Bu alanlarda küçük tutarsızlıklar bile ürüne olan güveni azaltır. Yapılan çalışma özellikle şu hissi iyileştirmeyi hedefledi:

Önceki İzlenim	Yeni İzlenim
Bazı işlemler tarayıcı onayıyla, bazıları uygulama modaliyle ilerliyordu	Tüm kritik onaylar aynı ürün standardıyla ilerliyor
Silme/pasifleştirme aksiyonları bazen az açıklamalıydı	İşlemin sonucu açıkça yazılıyor
Ayarlar ekranları birbirinden kopuk davranıyordu	Ortak onay altyapısı ile davranış standardı başladı

Bu, özellikle kurulum sonrası işletme ayarları, yazıcı ayarları, kullanıcı yönetimi ve menü yönetimi gibi alanlarda daha profesyonel bir ürün hissi sağlar.

6. API Altyapısında Sağlamaştırma Yapıldı

Kullanıcının günlük kullanımda görmediği ama ürünün uzun vadeli kalitesini etkileyen bir altyapı iyileştirmesi de yapıldı.

Önceden API dosyası hem sunucuyla konuşma mantığını hem de tüm ürün endpointlerini aynı yerde taşıyordu. Bu yapı büyüdükçe bakım zorlaşıyor, test etmek ve domainlere ayırmak riskli hale geliyordu.

Bu turda HTTP iletişim çekirdeği ayrı bir dosyaya çıkarıldı. Mevcut ekranların kullandığı `api` arayüzü korunarak davranış kırılmadı.

Değişiklik	Neden Önemli?
HTTP/token/request çekirdeği ayrıldı	Gelecekte sipariş, masa, ödeme, yazıcı gibi API grupları daha rahat ayrılabilir
Mevcut <code>api.*</code> kullanımı korundu	Ekranlarda kırıcı değişiklik yapılmadı
Davranış korunarak mimari sınır açıldı	Büyük refactorlara daha güvenli geçiş zemini oluştu

7. Temizlenen Amatör İzler

Bu turda özellikle ürün kalitesini düşüren küçük ama görünür izler temizlendi.

Alan	Temizlenen Sorun
Native onay kutuları	<code>window.confirm</code> kullanımları kaldırıldı
Debug çıktıları	Yazıcı keşif ekranındaki gereksiz <code>console.log</code> çıktıları kaldırıldı
Tekrar eden ödeme hesabı	Ödeme hesapları ortak yapıya taşındı
Dağınık sipariş kararları	İlk karar katmanı sipariş policy dosyasına alındı
API monolit başlangıcı	HTTP çekirdeği servis dosyasından ayrıldı

8. Doğrulama Sonuçları

Uygulanan değişikliklerden sonra temel doğrulamalar çalıştırıldı.

Komut	Sonuç	Açıklama
<code>npm run build</code>	Başarılı	Frontend production build sorunsuz tamamlandı
<code>npm test</code>	Başarılı	Server testlerinde 12 test dosyası, 108 test geçti
<code>npm run lint --if-present</code>	Çalıştı, çıktı üretmedi	Repo içinde lint scripti tanımlı değil
Statik arama	Temiz	<code>window.confirm</code> ve debug <code>console.log</code> kalmadı

9. Bu Çalışmanın Ürün Değeri

Değer	Açıklama
Daha az hata riski	Ödeme ve sipariş kuralları tek merkezden yönetilmeye başlandı
Daha profesyonel görünüm	Kritik onaylar artık uygulamanın kendi tasarım diliyle gösteriliyor
Daha güvenli operasyon	Silme, pasifleştirme, çıkış ve sıfırlama işlemleri daha açıklamalı hale geldi
Daha kolay bakım	Büyük dosyaları parçalamaya başlamadan önce güvenli ortak katmanlar kuruldu
Daha iyi test zemini	Saf yardımcı fonksiyonlar ileride kolayca unit test kapsamına alınabilir

10. Kalan Açık Alanlar

Bu tur kalıcı ve sağlam bir temel attı; ancak daha büyük frontend mimari iyileştirmeleri kontrollü şekilde sonraki turlara bırakılmalı.

Kalan Alan	Neden Hemen Tamamlandı?	Önerilen Sonraki Adım
<code>OrderScreen</code> parçalama	Çok yoğun ekran; tek hamlede bölmek regresyon riski doğurur	Katalog, sepet, müşteri ve işlem hookları ayrı ayrı çıkarılmalı
<code>TablesScreen</code> parçalama	Masa, paket, transfer, caller ID ve refresh akışı aynı ekranda	<code>useTablesData</code> , <code>TakeawaySidebar</code> , <code>TableCard</code> ayrımı yapılmalı
<code>PrinterDetailPage</code> parçalama	Yazıcı formu, keşif, önizleme ve encoding aynı dosyada	Form modelini hook'a, önizlemeyi ayrı bileşene almak
Test altyapısı	Frontend için özel test scripti yok	Utility unit testleri ve Playwright smoke testleri eklenmeli
Lint standardı	Repo lint scripti tanımlı değil	ESLint/format kontrolü standart build pipeline'a eklenmeli

11. Sonraki Geliştirme Planı

Hemen Devam Edilecekler

Öncelik	İş	Beklenen Kazanç
P1	Sipariş ekranını küçük hooklara ayırma	Daha az kırılgan sipariş geliştirmesi
P1	Masa ekranı veri yenileme mantığını merkezi hook'a alma	Socket/polling/manuel refresh karmaşasını azaltma
P1	Frontend utility testleri ekleme	Ödeme ve sipariş kurallarını otomatik koruma

Bu Hafta Ele Alınacaklar

Öncelik	İş	Beklenen Kazanç
P2	Yazıcı detay ekranını form ve önizleme parçalarına ayırma	Yazıcı ayarlarında bakım kolaylığı
P2	API servislerini domain gruplarına bölmeye devam etme	Daha temiz entegrasyon mimarisi
P2	Settings shell / ortak ayarlar layout'u	Yönetim ekranlarında daha tutarlı ürün hissi

Sonraya Bırakılacaklar

Öncelik	İş	Gerekçe
P2	React Query veya hafif cache katmanı	Daha geniş veri mimarisi kararı gerektirir
P2	Route state yerine URL/server hydrate modeli	Davranış etkisi yüksek, planlı yapılmalı
P2	Inline style borcunun tasarım sistemine taşınması	Görsel regresyon testiyle ilerlemeli

12. Kısa Sonuç

Bu turda uygulamanın görünmeyen ama güvenilirliği belirleyen temeli güçlendirildi. Ödeme, sipariş ve masa davranışları daha merkezi hale geldi; yönetim ekranlarındaki kritik onaylar profesyonel bir standarda kavuştu; API tarafında daha temiz bir mimari sınır açıldı.

Ürün artık bir sonraki geliştirme turuna daha sağlam bir zeminden devam edebilir. En doğru devam adımı, büyük ekranları küçük ama güvenli parçalara ayırmak ve bu kuralları otomatik testlerle koruma altına almak olacaktır.